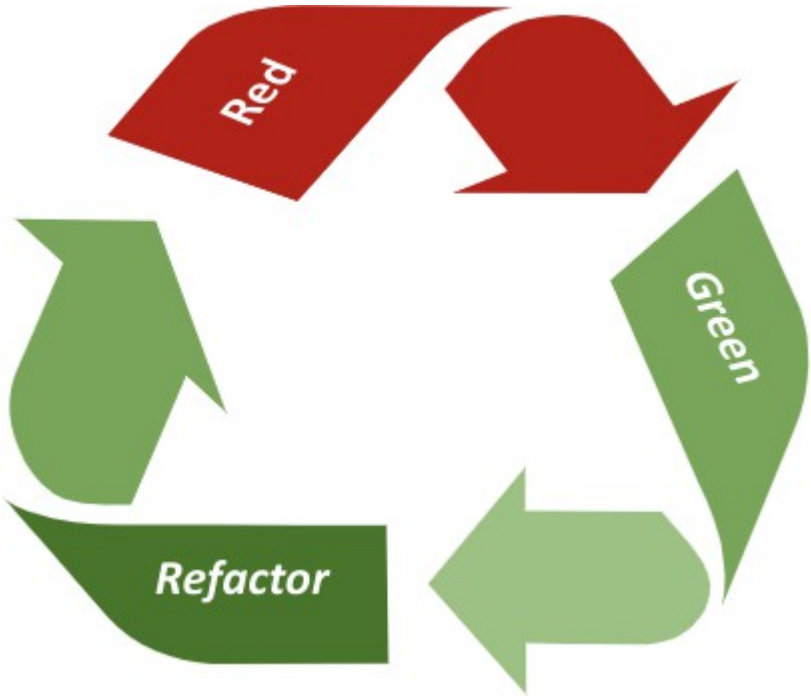
DeepL

Subscribe to DeepL Pro to translate larger documents.
Visit www.DeepL.com/pro for more information.

Guided Exercise 3: Problem Statement



Refactoring and Simple Design

Software Development

April 2026

Bachelor's Degree in Computer Engineering – Double Degree in Computer Science and Business Administration

Table of Contents

1. OBJECTIVES AND CASE STUDY *3*

2. REFACTORING *4*

Step2.1: Identification and resolution of refactoring situations **4**

2.2: Step Updating the code in accordance with coding standards **6**

Step2.3: Execution of defined test cases **6**

Step2.4: Committing the functional and test code to GitHub **6**

3. SIMPLE DESIGN *7*

Step3.1: : Applying the Singleton pattern to ensure that only one instance of a class can be created **7**

Step3.2: Uploading the functional and test code to GitHub **9**

4. Posts related to refactoring. *9*

5. TASKS TO BE PERFORMED *10*

6. RULES AND PROCEDURES *11*

1. OBJECTIVES AND CASE STUDY

The objectives of this guided exercise are:

- To put the refactoring technique into practice.
- To put into practice the principles of simple design and basic design patterns.

As a starting point for this guided exercise, you will be provided with source code that successfully implements some of the functionalities related to those presented in the previous guided exercise; therefore, as a first step, you will need to ensure that the defined test cases allow the source code to be verified satisfactorily. It is very important to bear in mind that the code provided implements some of the functionalities described in the statement for guided exercise 2 without any adaptations that may have been made during the practical sessions. A new function has also been added that retrieves documents for a specific date. This function accepts any date in a valid format and returns the number of documents found, as well as a record in a log file. Basic tests are included to verify this functionality during the refactoring process.

Firstly, the code must be reorganised in such a way that various predefined refactoring scenarios—which have been intentionally introduced into the code to facilitate learning this technique—can be resolved. The cases proposed for applying this technique are provided in Section 2.

Subsequently, various improvements to the proposed technical architecture for the provided code will be addressed, enabling the application of the fundamental principles of simple design and some basic design patterns. Recommendations for applying patterns are described in Chapter 3 of this statement.

This exercise does not have a final assignment document: therefore**, it is mandatory to commit and push after each step of the refactoring process, including a meaningful comment on the changes made**.

2. REFACTORING

The following situations requiring reorganisation have been identified in the source code of the component being developed by UC3M Consulting. Code reorganisation tasks must be carried out in the order proposed below, and at the end of each stage, it must be verified that the tests continue to function correctly. Should any test case need updating, you must first consult with the internship supervisor.

Step 2.1: Identification and resolution of refactoring situations.

The refactoring situations intentionally included in the code are as follows:

a) Mysterious names. Writing cryptic text is fine when you're writing or reading a detective novel, but not when you're reading code. One of the most important aspects of clear code is good naming, which is why we put a lot of thought into the names of functions, modules, variables and classes, so that they clearly communicate what they do and how to use them. The most common refactorings in this case are renames: Change Function Declaration (to rename a function), Rename Variable and Rename Field.

b) If you see the same code structure in more than one place, you can be sure that your programme will be better if you find a way to unify them. Duplication means that every time you read these copies, you'll have to read them carefully to see if there are any differences. If you need to change the duplicate code, you must find and capture each instance of duplication. The simplest problem with duplicate code is when you have the same expression in two methods of the same class. Then all you need to do is Extract Function and call the code from both places. If you have similar code, but not entirely identical, see if you can use Slide Statements to organise the code so that similar elements are all grouped together for easy extraction. If the duplicate fragments are in subclasses of a common superclass, you can use the

Pull Up method to avoid one calling the other.

c) Long Functions. In our experience, programmes that last longer and have fewer problems are those with short functions or methods. Since the early days of programming, programmers have realised that the longer a function is, the harder it is to understand. Ninety-nine per cent of the time, all you need to do to shorten a function is to extract it. Find parts of the function that go well together and create a new one.

d) Divergent Changes. We structure our software to facilitate change; after all, software is meant to be easily adaptable. When we make a change, we want to be able to jump to a clear point in the system and make the change. Divergent change occurs when a module is frequently changed in different ways for different reasons. If you look at a module and say: “Well, I’ll have to change these three functions every time I rename a file; I have to change these four functions every time a new type of financial exchange appears”, this is an indication of divergent change. If the two aspects form a sequence—for example, first retrieving data from the database and then applying financial processing—then Split Phase separates the two with a clear data structure between them. If the functions combine both types of processing, use Extract Function to separate them before moving them. If the modules are classes, then Extract Class helps formalise how to make the division.

e) Large classes. When a class is trying to do too much, it often ends up with too many fields and methods. When a class has too many fields, code duplication is never far away. You can extract the class to group a number of variables together. Choose variables to group together in the new class that make sense. If inheritance makes sense in the situation, extracting a superclass or replacing type code with subclasses (which is essentially extracting a subclass) is often easier.

f) Comments. The reason we mention comments here is because it often happens that you come across code with copious comments and

realise that the comments are there because the code is bad. Comments lead us to flawed code that has all the negative defects that should lead to refactoring. When we're done, we often find that the comments are superfluous. If you need a comment to explain what a block of code does, try Extract Function. If the method is already extracted but you still need a comment to explain what it does, use Rename Function to change the name.

Step 2.2: Updating the code in accordance with coding guidelines.

Finally, the necessary updates must be made to ensure that the source code correctly incorporates the rules and recommendations defined in the PEP8 coding guidelines, validated via pyLint. The code provided has been cleaned up, but still contains some errors that must be resolved during the refactoring process.

Step 2.3: Running the defined test cases.

Finally, it must be verified that the result of the refactoring is satisfactory and meets the set expectations. To this end, the tests created must be re-run to verify the correct functioning of the EnterpriseManager component.

It is important to note that this step must be carried out immediately after the completion of each code reorganisation task. For example, if a method is extracted, the tests must be run immediately afterwards to check that the programme's behaviour has not changed.

If you need to update any test cases as a result of the refactoring process, you must consult your tutor first.

Step 2.4: Uploading the functional code and tests to GitHub

It is mandatory that, after each change, once you have verified that the tests are still running correctly, you commit the changes, including a brief description of the change made in the commit message. This will serve as the documentation for your assignment, as the tutor will be able to see, step by step, the changes

made.

As completing the guided exercise will involve several work sessions, each session must end with the successful execution of the test cases and the submission of the corresponding code.

Code commits containing a large number of changes are not permitted. An extreme example would be a repository where the first commit is the original code and there is only one subsequent commit after which all the code is updated.

3. SIMPLE DESIGN

This section will address various improvements to the technical architecture proposed for the solution to guided exercise 2, which allow the fundamental principles of simple design and some basic design patterns to be applied.

Only one instance of the EnterpriseManager class and of the classes created to manage each data file may be run. To achieve this, the Singleton pattern will be implemented.

Step 3.1: Applying the Singleton pattern to ensure that only one instance of a class can be created

In software engineering, a singleton is a design pattern that restricts the creation of objects belonging to a class or the value of a type to a single object.

Its purpose is to ensure that a class has only one instance and to provide a global access point to it.

The singleton pattern is implemented by creating a method within our class that creates an instance of the object only if one does not already exist. To ensure that the class cannot be instantiated again, the scope of the constructor is restricted (using access modifiers such as protected or private).

Below is an example of the Python code required to

implement the singleton pattern [1] :

```
class IAmUnique(object):

     class  IAmUnique:

       def  init (self): self.name =

            None

       def str(self):

             return `self` + ' ' + self.name instance

      = None



    def new(cls):

       if not SoyUnico.instance:

            SoyUnico.instance = SoyUnico.SoyUnico()

       return SoyUnico.instance



    def getattr(self, name):

       return getattr(self.instance, name)



    def setattr(self, name, value):

       return setattr(self.instance, name, value)
```

[1] Example taken from https://jarroba.com/patron-singleton-python-ejemplos/

Step 3.2: Committing functional and test code to GitHub

It is mandatory that, after each change, once correct operation has been verified by running the tests, the code is pushed to the repository. For example, after applying the singleton pattern to a class and verifying that the tests run correctly, the code should be pushed to the repository. Include a comment in each commit describing the changes made: this will serve as the documentation for your exercise.

4. Publications related to refactoring.

Beyond practical application, research is being conducted into the application of refactoring techniques. For example, the article: "Energy Efficiency Analysis of Code Refactoring Techniques for Green and Sustainable Software in Portable Devices" (https://www.mdpi.com/2079-9292/11/3/442) analyses the impact of refactoring on energy efficiency.

Conduct a search on Google Scholar (https://scholar.google.com) for publications related to refactoring. In a single Word document, **each** team **member** should include the details (title, name of the journal or conference where it was published, web address where it can be found, authors and publication date) of **two articles** published within the last two years. Each student will include a **brief summary** of the articles (**one paragraph** for each) briefly explaining why they consider them relevant.

5. TASKS TO BE COMPLETED

1. **Create a new repository on GitHub for this guided exercise**. Name this repository according to the rules described in guided exercise 2 (**GXX.2026.TYY.EG3**) and **invite your tutor**.
2. Clone the project and include the code and folder structure available on Aula Global.
3. Apply the necessary refactoring and design techniques, both those mentioned and those identified in the various practical sessions.
4. **You must upload the refactored and redesigned code after each step in the process**. You must record each change with a commit, so that **the commit history shows the process followed step by step**. This will serve as the documentation for your project**. You will not pass if you do not comply with this section**.
5. The code must comply with the PEP8 standard as verified by the pyLint tool.
6. Search for the articles and include the document containing the references and abstracts of the articles in a folder called “doc”.
7. Include in the final submission for the practical assignment the latest XML report generated by pyBuilder following the execution of all test cases. You can run `pyb clean` before running `pyb` for the last time to easily locate the latest report. Save this report in the “doc” folder.
8. From GitHub, download the zip file containing the full project content and upload the zip file to the Aula Global submission portal enabled by the practical course tutor.

6. RULES AND PROCEDURES

This exercise is to be completed in pairs. If a student fails to submit any work, or if it is found that the number of submissions containing significant changes is very low compared to their partner's, they may be awarded a mark of 0 points. By 'significant changes', we mean those related to the proposed refactoring process. An example of a non-significant change would be a commit in which a space has been added within a comment and no other element of the code has been changed. **Similarly, failing to commit and push with comments after each change may result in a score of 0 points.**

The deadline for this exercise is 24 April 2026 before 23:59.

In accordance with the continuous assessment standards established for this course, if a team does not publish the solution to the exercise before the deadline, the exercise will be assessed with a score of 0 points.